DEUX PROGRAMMES ACTIFS

Réseau intelligent

et résilience applicative avec SAWM & Basira

Envoyer, conserver et reprendre des messages après une interruption ; comparer un ordonnancement par moyenne glissante avec un classement ODVL. Deux programmes autonomes, reliés aux véritables API Cloud et livrés avec leur code.

20/20	1080	0
Messages repris et livrés	Réceptions HTTP vérifiées	Erreur de livraison du banc

Résilience applicative. Une file SQLite survit au redémarrage de l'émetteur. Les accusés de réception lient l'identifiant au contenu ; une nouvelle tentative ne produit pas un second effet.

Réseau intelligent. Une file de messages HTTP utilise une moyenne glissante exponentielle (EWMA), l'ordre ODVL ou leur combinaison. Le benchmark conserve cinq politiques et trois scénarios, sans changer la concurrence.

Conclusion mesurée. La reprise fonctionne sur le scénario essayé. En revanche, EWMA + ODVL ne domine pas les méthodes simples sur ces charges : le raccordement fonctionne, son avantage de performance n'est pas démontré.

Portée. Échanges TCP/HTTP réels sur localhost, charges synthétiques sur un Mac ARM64. Aucun essai de radio, de GNSS, de localisation de brouilleur ou de résistance au brouillage physique. Le premier programme constitue une alternative civile de continuité applicative.

ARCHITECTURE VÉRIFIÉE

Le Cloud, puis l'exécution locale

Les entités ont été créées dans l'espace client Basira depuis Chrome. Le principal existant a reçu, après autorisation, lecture et écriture sur les deux nouvelles entités ; aucun droit d'administration ou de purge n'a été ajouté.

Étape	Rôle effectif	Vérification
1. Basira	Conserver les mesures de calibration puis le bilan.	Dépôt accepté, traitement terminé.
2. TSST	Retrouver le résumé du même essai dans la même entité.	Correspondance littérale et intégrité des preuves.
3. ODVL	Classer les candidats fournis avant le banc.	Permutation exacte, aucun candidat ajouté ou perdu.
4. SAWM	Proposer une politique JSON bornée ; commenter le bilan.	Tour terminé, paramètres validés localement.
5. Programme	Exécuter la politique, mesurer et vérifier les réceptions.	Identifiants, empreintes et compteurs locaux.

20 opérations Cloud suivies, toutes terminées lors de cette campagne. Ce nombre désigne les opérations du client (l'attente d'un traitement peut effectuer plusieurs requêtes HTTP), pas un compteur brut de requêtes réseau.

La mémoire rappelée a réellement alimenté la demande à SAWM. SAWM a retenu alpha = 0,35 pour l'EWMA et la politique de reprise proposée. Le choix identique à l'exemple fourni ne démontre pas une optimisation apprise.

Les appels Cloud sont placés avant et après les essais. La boucle locale n'attend pas le LLM pour chaque message. Un fichier de politique valide et un ordre ODVL déjà reçu peuvent être réutilisés hors ligne. Cette séparation protège le débit applicatif, sans prouver un réseau autonome universel.

PROGRAMME 1

Reprendre sans perdre les messages

Même ensemble de 20 messages, même destinataire HTTP et même coupure de 2.009 s. L'émetteur durable est arrêté puis relancé pendant l'indisponibilité. Un accusé de réception est volontairement perdu après l'écriture côté destinataire.

Contrôle	Résultat
Réceptions uniques / ACK vérifiés	20 / 20
Messages perdus en fin d'essai	0
Requêtes répétées / effets en double	1 / 0
Redémarrage du processus émetteur	Effectué et vérifié
Durée totale du scénario	2.850 s

Ce qui produit la reprise : journal SQLite persistant, nouvelles tentatives bornées, vérification de l'accusé de réception et idempotence au destinataire. Le même mécanisme local avait déjà livré 20/20 messages sans Cloud lors de la calibration.

Basira et TSST ajoutent ici conservation et rappel des résultats ; SAWM fournit une politique contrôlée ; ODVL classe des options. Aucun gain causal de livraison supplémentaire dû au Cloud n'est établi par ce test. La liaison n'est pas restée connectée pendant la coupure : les messages ont été conservés puis livrés après son retour.

PROGRAMME 2 - LATENCE

Comparer, sans présumer du résultat

P95 de latence de bout en bout, en millisecondes : médiane des trois répétitions. Plus bas est préférable. Chaque essai traite 24 messages avec au maximum quatre requêtes simultanées.

Charge régulière

Arrivées en rafales

Service ralenti

Une latence plus basse peut masquer davantage de messages hors délai. Le tableau suivant donne ce second critère. Ces mesures dépendent de l'ordonnanceur du Mac et de charges déterministes ; trois répétitions ne fondent pas une généralisation statistique.

ARBITRAGE

Débit, délais et limites

Messages hors délai sur 72 messages par scénario et politique (24 × 3 répétitions). Le contenu des messages reste livré et vérifié, même lorsqu'une échéance est dépassée.

Politique	Régulier	Rafales	Ralenti
FIFO	0	8	48
Échéance	0	0	30
EWMA	0	0	32
ODVL	0	15	42
EWMA + ODVL	0	6	44

Débit médian (messages/s)	Régulier	Rafales	Ralenti
FIFO	115.0	108.6	55.6
Échéance	112.9	100.4	55.1
EWMA	112.4	100.9	50.8
ODVL	112.1	113.2	56.5
EWMA + ODVL	114.0	101.7	54.7

Lecture. Sur les rafales, ODVL seul améliore le P95 face à FIFO dans ce banc, mais dépasse davantage d'échéances. EWMA seule et le tri par échéance évitent ici les retards. L'hybride ne domine ni l'un ni l'autre. Sur le service ralenti, l'hybride conserve plus de retards que le tri par échéance.

Amélioration à évaluer : définir l'objectif métier (délais, débit, équité), calibrer le poids ODVL sur des charges distinctes, puis figer la politique avant des tests sur des traces nouvelles. Ne pas ajuster sur ce jeu et le réutiliser comme preuve indépendante.

GUIDE DÉVELOPPEUR

Installer, reproduire, examiner

1. Dans l'espace Cloud. Créer une entité de laboratoire par programme. Autoriser son principal en lecture et écriture sur ces seules entités. La clé doit permettre les dépôts, la lecture de jobs, TSST, ODVL et SAWM. Conserver les secrets dans un fichier privé, hors Git.

2. Installer le code. Chaque dépôt contient README, exigences, tests, SDK Cloud officiel, politique bornée et preuves mesurées. Python suffit ; aucun pilote radio, accès root ni changement du réseau de l'hôte n'est requis.

Programme	Point d'entrée
Résilience applicative	lab.py --messages 20 --policy POLICY.json --output-dir NOUVEAU_DOSSIER
Réseau intelligent	queue_engine.py --count 24 --repetitions 3 --orders ORDERS.json --policy POLICY.json --output RESULTAT.json
Raccordement Cloud commun	cloud_control.py prepare / record --kind network / delivery --run ...

3. Examiner les preuves. Les rapports JSON gardent les politiques, les réceptions, les empreintes, l'ordre des essais et les opérations Cloud. Une empreinte relie les fichiers ; elle n'est pas une signature indépendante du serveur Cloud. Les échecs de validation ne sont pas convertis en succès.

4. Séparer les usages. Les profils de trafic sont synthétiques, les échanges HTTP et les redémarrages sont réels. Ces programmes ne configurent ni réseau opérateur, ni véhicule, ni système radio. Le dimensionnement industriel, les vrais liens distants et les garanties de disponibilité restent à évaluer.

Dépôts :
github.com/ulookme/sawm-reseau-intelligent
github.com/ulookme/sawm-resilience-applicative

Sources du rapport : network-benchmark.json ; delivery-benchmark/report.json ; quatre cloud-report.json de calibration et de bilan. Version de campagne : 27 octobre 2026. Voir les README pour les commandes complètes et les identifiants de commit.